

Chapter 16. Accessing Files

Accessing a disk-based file is a complex activity that involves the VFS abstraction layer ([Chapter 12](#)), handling block devices ([Chapter 14](#)), and the use of the page cache ([Chapter 15](#)). This chapter shows how the kernel builds on all those facilities to carry out file reads and writes. The topics covered in this chapter apply both to regular files stored in disk-based filesystems and to block device files; these two kinds of files will be referred to simply as "files."

The stage we are working at in this chapter starts after the proper read or write method of a particular file has been called (as described in [Chapter 12](#)). We show here how each read ends with the desired data delivered to a User Mode process and how each write ends with data marked ready for transfer to disk. The rest of the transfer is handled by the facilities described in [Chapter 14](#) and [Chapter 15](#).

There are many different ways to access a file. In this chapter we will consider the following cases:

Canonical mode

The file is opened with the `O_SYNC` and `O_DIRECT` flags cleared, and its content is accessed by means of the `read( )` and `write( )` system calls. In this case, the `read( )` system call blocks the calling process until the data is copied into the User Mode address space (however, the kernel is always allowed to return fewer bytes than requested!). The `write( )` system call is different, because it terminates as soon as the data is copied into the page cache (deferred write). This case is covered in the section "[Reading and Writing a File](#)."

Synchronous mode

The file is opened with the `O_SYNC` flag set or the flag is set at a later time by the `fcntl( )` system call. This flag affects only the write operation (read operations are always blocking), which blocks the calling process until the data is effectively written to disk. The section "[Reading and Writing a File](#)" covers this case, too.

Memory mapping mode

After opening the file, the application issues an `mmap( )` system call to map the file into memory. As a result, the file appears as an array of bytes in RAM, and the application accesses directly the array elements instead of using `read( )`, `write( )`, or `lseek( )`. This case is discussed in the section "[Memory Mapping](#)."

Direct I/O mode

The file is opened with the `O_DIRECT` flag set. Any read or write operation transfers data directly from the User Mode address space to disk, or vice versa, bypassing the page cache. We discuss this case in the section "[Direct I/O Transfers](#)." (The values of the `O_SYNC` and `O_DIRECT` flags can be combined in four meaningful ways.)

Asynchronous mode

The file is accessed either through a group of POSIX APIs or by means of Linux-specific system calls in such a way to perform "asynchronous I/O:" this means the requests for data transfers never block the calling process; rather, they are carried on "in the background" while the application

continues its normal execution. We discuss this case in the section "[Asynchronous I/O](#)."

16.1. Reading and Writing a File

The section "[The read\(\) and write\(\) System Calls](#)" in [Chapter 12](#) described how the `read( )` and `write( )` system calls are implemented. The corresponding service routines end up invoking the file object's `read` and `write` methods, which may be filesystem-dependent. For disk-based filesystems, these methods locate the physical blocks that contain the data being accessed and activate the block device driver to start the data transfer.

Reading a file is page-based: the kernel always transfers whole pages of data at once. If a process issues a `read( )` system call to get a few bytes, and that data is not already in RAM, the kernel allocates a new page frame, fills the page with the suitable portion of the file, adds the page to the page cache, and finally copies the requested bytes into the process address space. For most filesystems, reading a page of data from a file is just a matter of finding what blocks on disk contain the requested data. Once this is done, the kernel fills the pages by submitting the proper I/O operations to the generic block layer. In practice, the `read` method of all disk-based filesystems is implemented by a common function named `generic_file_read( )`.

Write operations on disk-based files are slightly more complicated to handle, because the file size could increase, and therefore the kernel might allocate some physical blocks on the disk. Of course, how this is precisely done depends on the filesystem type. However, many disk-based filesystems implement their `write` methods by means of a common function named `generic_file_write( )`. Examples of such filesystems are Ext2, System V /Coherent /Xenix , and MINIX . On the other hand, several other filesystems, such as journaling and network filesystems , implement the `write` method by means of custom functions.

16.1.1. Reading from a File

The `generic_file_read( )` function is used to implement the `read` method for block device files and for regular files of almost all disk-based filesystems. This function acts on the following parameters:

`filp`

Address of the file object

buf	Linear address of the User Mode memory area where the characters read from the file must be stored
count	Number of characters to be read
ppos	Pointer to a variable that stores the offset from which reading must start (usually the <code>f_pos</code> field of the <code>filp</code> file object)

As a first step, the function initializes two descriptors. The first descriptor is stored in the local variable `local_iov` of type `iovec`; it contains the address (`buf`) and the length (`count`) of the User Mode buffer that shall receive the data read from the file. The second descriptor is stored in the local variable `kiocb` of type `kiocb`; it is used to keep track of the completion status of an ongoing synchronous or asynchronous I/O operation. The main fields of the `kiocb` descriptor are shown in [Table 16-1](#).

Table 16-1. The main fields of the `kiocb` descriptor

Type	Field	Description
<code>struct list_head</code>	<code>ki_run_list</code>	Pointers for the list of I/O operations to be retried later
<code>long</code>	<code>ki_flags</code>	Flags of the <code>kiocb</code> descriptor
<code>int</code>	<code>ki_users</code>	Usage counter of the <code>kiocb</code> descriptor
<code>unsigned int</code>	<code>ki_key</code>	Identifier of the asynchronous I/O operation, or <code>KIOCB_SYNC_KEY</code> (<code>0xffffffff</code>) for synchronous I/O operations
<code>struct file *</code>	<code>ki_filp</code>	Pointer to the file object associated with the ongoing I/O operation
<code>struct kiocx *</code>	<code>ki_ctx</code>	Pointer to the asynchronous I/O context descriptor for this operation (see the section " Asynchronous I/O " later in this chapter)
<code>int (*)</code> <code>(struct kiocb *,</code> <code>struct io_event *)</code> <code>ssize_t (*)</code>	<code>ki_cancel</code>	Method invoked when canceling an asynchronous I/O operation
<code>(struct kiocb *)</code>	<code>ki_retry</code>	Method invoked when retrying an asynchronous I/O operation

<code>void (*)</code>		Method invoked when destroying the <code>kiocb</code> descriptor
<code>(struct kiocb *)</code>	<code>ki_dtor</code>	Pointers for the list of active ongoing I/O operation on an asynchronous I/O context
<code>struct list_head</code>	<code>ki_list</code>	For synchronous operations, pointer to the process descriptor that issued the I/O operation; for asynchronous operations, pointer to the <code>ioctx</code> User Mode data structure
<code>union</code>	<code>ki_obj</code>	Value to be returned to the User Mode process
<code>__u64</code>	<code>ki_user_data</code>	Current file position of the ongoing I/O operation
<code>loff_t</code>	<code>ki_pos</code>	Type of operation (read, write, or sync)
<code>unsigned short</code>	<code>ki_opcode</code>	Number of bytes to be transferred
<code>size_t</code>	<code>ki_nbytes</code>	Current position in the User Mode buffer
<code>char *</code>	<code>ki_buf</code>	Number of bytes yet to be transferred
<code>size_t</code>	<code>ki_left</code>	Wait queue used for asynchronous I/O operations
<code>wait_queue_t</code>	<code>ki_wait</code>	Freely usable by the filesystem layer
<code>void *</code>	<code>private</code>	

The `generic_file_read( )` function initializes the `kiocb` descriptor by executing the `init_sync_kiocb` macro, which sets the fields of the object for a synchronous operation. In particular, the macro sets the `ki_key` field to `KIOCB_SYNC_KEY`, the `ki_filp` field to `filp`, and the `ki_obj` field to `current`.

Then, `generic_file_read( )` invokes `__generic_file_aio_read( )` passing to it the addresses of the `iovec` and `kiocb` descriptors just filled. The latter function returns a value, which is usually the number of bytes effectively read from the file; `generic_file_read( )` terminates by returning this value.

The `__generic_file_aio_read( )` function is a general-purpose routine used by all filesystems to implement both synchronous and asynchronous read operations. The function receives four parameters: the address `ioctx` of a `kiocb` descriptor, the address `iov` of an array of `iovec` descriptors, the length of this array, and the address `ppos` of a variable that stores the file's current pointer. When invoked by `generic_file_read( )`, the array of `iovec` descriptors is composed of just one element describing the User Mode buffer that will receive the data.^[*]

[*] A variant of the `read( )` system call named `readv( )` allows an application to define multiple User Mode buffers in which the kernel scatters the data read from the file; the `__generic_file_aio_read( )` function handles this case, too. In the following, we will assume that the data read from the file will be copied into just one User Mode buffer; however, guessing the additional steps to be performed when using multiple buffers is

straightforward.

We now explain the actions of the `__generic_file_aio_read()` function; for the sake of simplicity, we restrict the description to the most common case: a synchronous operation raised by a `read()` system call on a page-cached file. Later in this chapter we describe how this function behaves in other cases. As usual, we do not discuss how errors and anomalous conditions are handled.

Here are the steps performed by the function:

1. Invokes `access_ok()` to verify that the User Mode buffer described by the `iovec` descriptor is valid. Because the starting address and length have been received from the `sys_read()` service routine, they must be checked before using them (see the section "[Verifying the Parameters](#)" in [Chapter 10](#)). If the parameters are not valid, returns the `-EFAULT` error code.
2. Sets up a read operation descriptor namely, a data structure of type `read_descriptor_t` that stores the current status of the ongoing file read operation relative to a single User Mode buffer. The fields of this descriptor are shown in [Table 16-2](#).
3. Invokes `do_generic_file_read()`, passing to it the file object pointer `filp`, the pointer to the file offset `ppos`, the address of the just allocated read operation descriptor, and the address of the `file_read_actor()` function (see later).
4. Returns the number of bytes copied into the User Mode buffer; that is, the value found in the `written` field of the `read_descriptor_t` data structure.

Table 16-2. The fields of the read operation descriptor

Type

Field

Description

`size_t`

`written`

How many bytes have been copied into the User Mode buffer

`size_t`

`count`

How many bytes are yet to be transferred

`char *`

`arg.buf`

Current position in the User Mode buffer

`int`

`error`

Error code of the read operation (0 for no error)

The `do_generic_file_read( )` function reads the requested pages from disk and copies them into the User Mode buffer. In particular, the function performs the following actions:

1. Gets the `address_space` object corresponding to the file being read; its address is stored in `filp->f_mapping`.
2. Gets the owner of the `address_space` object, that is, the inode object that will own the pages to be filled with file's data; its address is stored in the `host` field of the `address_space` object. If the file being read is a block device file, the owner is an inode in the bdev special filesystem rather than the inode pointed to by `filp->f_dentry->d_inode` (see "[The address_space Object](#)" in [Chapter 15](#)).
3. Considers the file as subdivided in pages of data (4,096 bytes per page). The function derives from the file pointer `*ppos` the logical number of the page that includes the first requested byte that is, the page's index in the address space and stores it in the `index` local variable. The function also stores in the `offset` local variable the displacement inside the page of the first requested byte.
4. Starts a cycle to read all pages that include the requested bytes; the number of bytes to be read is stored in the `count` field of the `read_descriptor_t` descriptor. During a single iteration, the function transfers a page of data by performing the following substeps:
 - a. If `index*4096+offset` exceeds the file size stored in the `i_size` field of the inode object, it exits from the cycle and goes to step 5.
 - b. Invokes `cond_resched( )` to check the `TIF_NEED_RESCHED` flag of the current process and, if the flag is set, to invoke the `schedule( )` function.
 - c. If additional pages must be read in advance, it invokes `page_cache_readahead( )` to read them. We defer discussing read-ahead until the later section "[Read-Ahead of Files](#)."
 - d. Invokes `find_get_page( )` passing as parameters a pointer to the `address_space` object and the value of `index`; the function looks up the page cache to find the descriptor of the page that stores the requested data, if any.
 - e. If `find_get_page( )` returned a `NULL` pointer, the page requested is not in the page cache. In that case, it performs the following actions:
 1. Invokes `handle_ra_miss( )` to tune the parameters used by the read-ahead system.
 2. Allocates a new page.
 3. Inserts the descriptor of the new page into the page cache by invoking `add_to_page_cache( )`. Remember that this function sets the `PG_locked` flag of the new page.
 4. Inserts the descriptor of the new page into the LRU list by invoking `lru_cache_add( )` (see [Chapter 17](#)).
 5. Jumps to step 4j to start reading the file's data.
 - f. If the function has reached this point, the page is in the page cache. Checks the `PG_uptodate` flag; if it is set, the data stored in the page is up-to-date, hence there is no need to read it from disk: jumps to step 4m.
 - g. The data on the page is not valid, so it must be read from disk. The function gains exclusive access to the page by invoking the `lock_page( )` function. As described in the section "[Page Cache Handling Functions](#)" in [Chapter 15](#), `lock_page( )` suspends the current process if the `PG_locked` flag is already set, until that bit is cleared.
 - h. Now the page is locked by the current process. However, another process might have removed the page from the page cache right before the previous step; hence, it checks whether the mapping field of the page descriptor is `NULL`; in this case, it unlocks the page by invoking `unlock_page( )`, decreases its usage counter (it was increased by `find_get_page( )`), and jumps back to step 4a starting over with the same page.
 - i. If the function has reached this point, the page is locked and still present in the page cache.

- Checks the `PG_uptodate` flag again, because another kernel control path could have completed the necessary read between steps 4f and 4g. If the flag is set, it invokes `unlock_page( )` and jumps to step 4m to skip the read operation.
- j. Now the actual I/O operation can be started. Invokes the `readpage` method of the `address_space` object of the file. The corresponding function takes care of activating the I/O data transfer from the disk to the page. We discuss later what this function does for regular files and block device files.
 - k. If the `PG_uptodate` flag is still cleared, it waits until the page has been effectively read by invoking the `lock_page( )` function. The page, which was locked in step 4g, will be unlocked as soon as the read operation finishes. Therefore, the current process sleeps until the I/O data transfer terminates.
 - l. If `index` exceeds the file size in pages (this number is obtained by dividing the value of the `i_size` field of the inode object by 4,096), it decreases the page's usage counter, and exits from the cycle jumping to step 5. This case occurs when the file being read is concurrently truncated by another process.
 - m. Stores in the `nr` local variable the number of bytes in the page that should be copied into the User Mode buffer. This value is equal to the page size (4,096 bytes) unless either `offset` is not zero or this can happen only for the first or last page of requested data or the file does not contain all requested bytes.
 - n. Invokes `mark_page_accessed( )` to set the `PG_referenced` or the `PG_active` flag, hence denoting the fact that the page is being used and should not be swapped out (see [Chapter 17](#)). If the same page (or part thereof) is read several times in successive executions of `do_generic_file_read( )`, this step is executed only during the first read.
 - o. Now it is time to copy the data on the page into the User Mode buffer. To do this, `do_generic_file_read( )` invokes the `file_read_actor( )` function, whose address has been passed as a parameter. In turn, `file_read_actor( )` essentially executes the following steps:
 - 1. Invokes `kmap( )`, which establishes a permanent kernel mapping for the page if it is in high memory (see the section "[Kernel Mappings of High-Memory Page Frames](#)" in [Chapter 8](#)).
 - 2. Invokes `_copy_to_user( )`, which copies the data on the page in the User Mode address space (see the section "[Accessing the Process Address Space](#)" in [Chapter 10](#)). Notice that this operation might block the process because of page faults while accessing the User Mode address space.
 - 3. Invokes `kunmap( )` to release any permanent kernel mapping of the page.
 - 4. Updates the `count`, `written`, and `buf` fields of the `read_descriptor_t` descriptor.
 - p. Updates the `index` and `offset` local variables according to the number of bytes effectively transferred in the User Mode buffer. Typically, if the last byte in the page has been copied into the User Mode buffer, `index` is increased by one and `offset` is set to zero; otherwise, `index` is not increased and `offset` is set to the number of bytes in the page that have been copied into the User Mode buffer.
 - q. Decreases the page descriptor usage counter.
 - r. If the `count` field of the `read_descriptor_t` descriptor is not zero, there is other data to be read from the file: jumps to step 4a to continue the loop with the next page of data in the file.
- 5. All requested or available bytes have been read. The function updates the `filp->f_ra` read-ahead data structure to record the fact that data is being read sequentially from the file (see the later section "[Read-Ahead of Files](#)").
 - 6. Assigns to `*ppos` the value `index*4096+offset`, thus storing the next position where a sequential access is to occur for a future invocation of the `read( )` and `write( )` system calls.
 - 7. Invokes `update_atime( )` to store the current time in the `i_atime` field of the file's inode and to mark the inode as dirty, and returns.

16.1.1.1. The readpage method for regular files

As we saw, the `readpage` method is used repeatedly by `do_generic_file_read( )` to read individual pages from disk into memory.

The `readpage` method of the `address_space` object stores the address of the function that effectively activates the I/O data transfer from the physical disk to the page cache. For regular files, this field typically points to a wrapper that invokes the `mpage_readpage( )` function. For instance, the `readpage` method of the Ext3 filesystem is implemented by the following function:

```
int ext3_readpage(struct file *file, struct page *page)
{
    return mpage_readpage(page, ext3_get_block);
}
```

The wrapper is needed because the `mpage_readpage( )` function receives as its parameters the descriptor `page` of the page to be filled and the address `get_block` of a function that helps `mpage_readpage( )` find the right block. The wrapper is filesystem-specific and can therefore supply the proper function to get a block. This function translates the block numbers relative to the beginning of the file into logical block numbers relative to positions of the block in the disk partition (for an example, see [Chapter 18](#)). Of course, the latter parameter depends on the type of filesystem to which the regular file belongs; in the previous example, the parameter is the address of the `ext3_get_block( )` function. The function passed as `get_block` always uses a buffer head to store precious information about the block device (`b_dev` field), the position of the requested data on the device (`b_blocknr` field), and the block status (`b_state` field).

The `mpage_readpage( )` function chooses between two different strategies when reading a page from disk. If the blocks that contain the requested data are contiguously located on disk, then the function submits the read I/O operation to the generic block layer by using a single bio descriptor. In the opposite case, each block in the page is read by using a different bio descriptor. The filesystem-dependent `get_block` function plays the crucial role of determining whether the next block in the file is also the next block on the disk.

Specifically, `mpage_readpage( )` performs the following steps:

1. Checks the `PG_private` field of the page descriptor: if it is set, the page is a buffer page, that is, the page is associated with a list of buffer heads describing the blocks that compose the page (see the section "[Storing Blocks in the Page Cache](#)" in [Chapter 15](#)). This means that the page has already been read from disk in the past, and that the blocks in the page are not adjacent on disk: jumps to step 11 to read the page one block at a time.
2. Retrieves the block size (stored in the `page->mapping->host->i_blkbits` inode field), and computes two values required to access all blocks on that page: the number of blocks stored in the page and the file block number of the first block in the pagethat is, the index of the first block in the page relative to the beginning of the file.
3. For each block in the page, invokes the filesystem-dependent `get_block` function passed as a parameter to get the logical block number, that is, the index of the block relative to the beginning of the disk or partition. The logical block numbers of all blocks in the page are stored in a local array.
4. Checks for any anomalous condition that could occur while executing the previous step. In particular, if some blocks are not adjacent on disk, or some block falls inside a "file hole" (see the section "[File Holes](#)" in [Chapter 18](#)), or a block buffer has been already filled by the `get_block` function, then jumps to step 11 to read the page one block at a time.
5. If the function has reached this point, all blocks on the page are adjacent on disk. However, the page could be the last page of data in the file, hence some of the blocks in the page might not have an

- image on disk. If so, it fills the corresponding block buffers in the page with zeros; otherwise, it sets the `PG_mappedtodisk` flag of the page descriptor.
6. Invokes `bio_alloc( )` to allocate a new bio descriptor consisting of a single segment and to initialize its `bi_bdev` and `bi_sector` fields with the address of the block device descriptor and the logical block number of the first block in the page, respectively. Both pieces of information have been determined in step 3 above.
 7. Sets the `bio_vec` descriptor of the bio's segment with the initial address of the page, the offset of the first byte to be read (zero), and the total number of bytes to be read.
 8. Stores the address of the `mpage_end_io_read( )` function in the `bio->bi_end_io` field (see below).
 9. Invokes `submit_bio( )`, which sets the `bi_rw` flag with the direction of the data transfer, updates the `page_states` per-CPU variable to keep track of the number of read sectors, and invokes the `generic_make_request( )` function on the bio descriptor (see the section "[Issuing a Request to the I/O Scheduler](#)" in [Chapter 14](#)).
 10. Returns the value zero (success).
 11. If the function jumps here, the page contains blocks that are not adjacent on disk. If the page is up-to-date (`PG_uptodate` flag set), the function invokes `unlock_page( )` to unlock the page; otherwise, it invokes `block_read_full_page( )` to start reading the page one block at a time (see below).
 12. Returns the value zero (success).

The `mpage_end_io_read( )` function is the completion method of the bio; it is executed as soon as the I/O data transfer terminates. Assuming that there was no I/O error, the function essentially sets the `PG_uptodate` flag of the page descriptor, invokes `unlock_page( )` to unlock the page and to wake up any process sleeping for this event, and invokes `bio_put( )` to destroy the bio descriptor.

16.1.1.2. The readpage method for block device files

In the sections "[VFS Handling of Device Files](#)" in [Chapter 13](#) and "[Opening a Block Device File](#)" in [Chapter 14](#), we discussed how the kernel handles requests to open a block device file. We saw how the `init_special_inode( )` function sets up the device inode and how the `blkdev_open( )` function completes the opening phase.

Block devices use an `address_space` object that is stored in the `i_data` field of the corresponding block device inode in the `bdev` special filesystem. Unlike regular files whose `readpage` method in the `address_space` object depends on the filesystem type to which the file belongs the `readpage` method of block device files is always the same. It is implemented by the `blkdev_readpage( )` function, which calls `block_read_full_page( )`:

```
int blkdev_readpage(struct file * file, struct * page page)
{
    return block_read_full_page(page, blkdev_get_block);
}
```

As you can see, the function is once again a wrapper, this time for the `block_read_full_page( )` function. This time the second parameter points to a function that translates the file block number relative to the beginning of the file into a logical block number relative to the beginning of the block device. For block device files, however, the two numbers coincide; therefore, the `blkdev_get_block( )` function performs the following steps:

1. Checks whether the number of the first block in the page exceeds the index of the last block in the block device (this index is obtained by dividing the size of the block device stored in `bdev->bd_inode->i_size` by the block size stored in `bdev->bd_block_size`; `bdev` points to the descriptor of the block device). If so, it returns `-EIO` for a write operation, or zero for a read operation. (Reading beyond the end of a block device is not allowed, either, but the error code should not be returned here: the kernel could just be trying to dispatch a read request for the last data of a block device, and the corresponding buffer page is only partially mapped.)
2. Sets the `b_dev` field of the buffer head to `bdev`.
3. Sets the `b_blocknr` field of the buffer head to the file block number, which was passed as a parameter of the function.
4. Sets the `BH_Mapped` flag of the buffer head to state that the `b_dev` and `b_blocknr` fields of the buffer head are significant.

The `block_read_full_page( )` function reads a page of data one block at a time. As we have seen, it is used both when reading block device files and when reading pages of regular files whose blocks are not adjacent on disk. It performs the following steps:

1. Checks the `PG_private` flag of the page descriptor; if it is set, the page is associated with a list of buffer heads describing the blocks that compose the page (see the section "[Storing Blocks in the Page Cache](#)" in [Chapter 15](#)). Otherwise, the function invokes `create_empty_buffers( )` to allocate buffer heads for all block buffers included in the page. The address of the buffer head for the first buffer in the page is stored in the `page->private` field. The `b_this_page` field of each buffer head points to the buffer head of the next buffer in the page.
2. Derives from the file offset relative to the page (`page->index` field) the file block number of the first block in the page.
3. For each buffer head of the buffers in the page, it performs the following substeps:
 - a. If the `BH_Uptodate` flag is set, it skips the buffer and continues with the next buffer in the page.
 - b. If the `BH_Mapped` flag is not set and the block is not beyond the end of the file, it invokes the filesystem-dependent `get_block` function whose address has been passed as a parameter. For a regular file, the function looks in the on-disk data structures of the filesystem and finds the logical block number of the buffer relative to the beginning of the disk or partition. Conversely, for a block device file, the function regards the file block number as the logical block number. In both cases the function stores the logical block number in the `b_blocknr` field of the corresponding buffer head and sets the `BH_Mapped` flag.^[*]
 - c. Tests again the `BH_Uptodate` flag because the filesystem-dependent `get_block` function could have triggered a block I/O operation that updated the buffer. If `BH_Uptodate` is set, it continues with the next buffer in the page.
 - d. Stores the address of the buffer head in `arr` local array, and continues with the next buffer in the page.
4. If no file hole has been encountered in the previous step, the function sets the `PG_mappedtodisk` flag of the page.
5. Now the `arr` local array stores the addresses of the buffer heads that correspond to the buffers whose content is not up-to-date. If this array is empty, all buffers in the page are valid. So the function sets the `PG_uptodate` flag of the page descriptor, unlocks the page by invoking `unlock_page( )`, and returns.
6. The `arr` local array is not empty. For each buffer head in the array, `block_read_full_page( )` performs the following substeps:

[*] When accessing a regular file, the `get_block` function might not find the block if it falls in a "file hole" (see the section "[File Holes](#)" in [Chapter 18](#)). In this case, the function fills the block buffer with zeros and sets the `BH_Uptodate` flag of the buffer head.

- a. Sets the `BH_Lock` flag. If the flag was already set, the function waits until the buffer is released.
 - b. Sets the `b_end_io` field of the buffer head to the address of the `end_buffer_async_read( )` function (see below) and sets the `BH_Async_Read` flag of the buffer head.
7. For each buffer head in the `arr` local array, it invokes the `submit_bh( )` function on it, specifying the operation type `READ`. As we saw earlier, this function triggers the I/O data transfer of the corresponding block.
 8. Returns 0.

The `end_buffer_async_read( )` function is the completion method of the buffer head; it is executed as soon as the I/O data transfer on the block buffer terminates. Assuming that there was no I/O error, the function sets the `BH_Uptodate` flag of the buffer head and clears the `BH_Async_Read` flag. Then, the function gets the descriptor of the buffer page containing the block buffer (its address is stored in the `b_page` field of the buffer head) and checks whether all blocks in the page are up-to-date; if so, the function sets the `PG_uptodate` flag of the page and invokes `unlock_page( )`.

16.1.2. Read-Ahead of Files

Many disk accesses are sequential. As we will see in [Chapter 18](#), regular files are stored on disk in large groups of adjacent sectors, so that they can be retrieved quickly with few moves of the disk heads. When a program reads or copies a file, it often accesses it sequentially, from the first byte to the last one. Therefore, many adjacent sectors on disk are likely to be fetched when handling a series of a process's read requests on the same file.

Read-ahead consists of reading several adjacent pages of data of a regular file or block device file before they are actually requested. In most cases, read-ahead significantly enhances disk performance, because it lets the disk controller handle fewer commands, each of which refers to a larger chunk of adjacent sectors. Moreover, it improves system responsiveness. A process that is sequentially reading a file does not usually have to wait for the requested data because it is already available in RAM.

However, read-ahead is of no use when an application performs random accesses to files; in this case, it is actually detrimental because it tends to waste space in the page cache with useless information. Therefore, the kernel reduces or stops read-ahead when it determines that the most recently issued I/O access is not sequential to the previous one.

Read-ahead of files requires a sophisticated algorithm for several reasons:

- Because data is read page by page, the read-ahead algorithm does not have to consider the offsets inside the page, but only the positions of the accessed pages inside the file.
- Read-ahead may be gradually increased as long as the process keeps accessing the file sequentially.
- Read-ahead must be scaled down or even disabled when the current access is not sequential with respect to the previous one (random access).
- Read-ahead should be stopped when a process keeps accessing the same pages over and over again (only a small portion of the file is being used), or when almost all pages of the file are already in the page cache.
- The low-level I/O device driver should be activated at the proper time, so that the future pages will have been transferred when the process needs them.

The kernel considers a file access as sequential with respect to the previous file access if the first page requested is the page following the last page requested in the previous access.

While accessing a given file, the read-ahead algorithm makes use of two sets of pages, each of which corresponds to a contiguous portion of the file. These two sets are called the current window and the ahead window .

The current window consists of pages requested by the process or read in advance by the kernel and included in the page cache. (A page in the current window is not necessarily up-to-date, because its I/O data transfer could be still in progress.) The current window contains both the last pages sequentially accessed by the process and possibly some of the pages that have been read in advance by the kernel but that have not yet been requested by the process.

The ahead window consists of pages following the ones in the current window that are being currently being read in advance by the kernel. No page in the ahead window has yet been requested by the process, but the kernel assumes that sooner or later the process will request them.

When the kernel recognizes a sequential access and the initial page belongs to the current window, it checks whether the ahead window has already been set up. If not, the kernel creates a new ahead window and triggers the read operations for the corresponding pages. In the ideal case, the process still requests pages from the current window while the pages in the ahead window are being transferred. When the process requests a page included in the ahead window, the ahead window becomes the new current window.

The main data structure used by the read-ahead algorithm is the `file_ra_state` descriptor whose fields are listed in [Table 16-3](#). Each file object includes such a descriptor in its `f_ra` field.

Table 16-3. The fields of the `file_ra_state` descriptor

Type

Field

Description

`unsigned long`

`start`

Index of first page in the current window

`unsigned long`

`size`

Number of pages included in the current window (–1 for read-ahead temporarily disabled, 0 for empty current window)

`unsigned long`

`flags`

Flags used to control the read-ahead

`unsigned long`

`cache_hit`

Number of consecutive cache hits (pages requested by the process and found in the page cache)

`unsigned long`

`prev_page`

Index of the last page requested by the process

`unsigned long`

`ahead_start`

Index of the first page in the ahead window

`unsigned long`

`ahead_size`

Number of pages in the ahead window (0 for an empty ahead window)

`unsigned long`

`ra_pages`

Maximum size in pages of a read-ahead window (0 for read-ahead permanently disabled)

`unsigned long`

`mmap_hit`

Read-ahead hit counter (used for memory mapped files)

`unsigned long`

`mmap_miss`

Read-ahead miss counter (used for memory mapped files)

When a file is opened, all the fields of its `file_ra_state` descriptor are set to zero except the `prev_page` and `ra_pages` fields.

The `prev_page` field stores the index of the last page requested by the process in the previous read operation; initially, the field contains the value `-1`.

The `ra_pages` field represents the maximum size in pages for the current window, that is, the maximum read-ahead allowed for the file. The initial (default) value for this field is stored in the `backing_dev_info` descriptor of the block device that includes the file (see the section "[Request Queue Descriptors](#)" in [Chapter 14](#)). An application can tune the read-ahead algorithm for a given opened file by modifying the `ra_pages` field; this can be done by invoking the `posix_fadvise( )` system call, passing to it the commands `POSIX_FADV_NORMAL` (set read-ahead maximum size to default, usually 32 pages), `POSIX_FADV_SEQUENTIAL` (set read-ahead maximum size to two times the default), and `POSIX_FADV_RANDOM` (set read-ahead maximum size to zero, thus permanently disabling read-ahead).

The `flags` field contains two flags called `RA_FLAG_MISS` and `RA_FLAG_INCACHE` that play an important role. The first flag is set when a page that has been read in advance is not found in the page cache (likely because it has been reclaimed by the kernel in order to free memory; see [Chapter 17](#)): in this case, the size of the next ahead window to be created is somewhat reduced. The second flag is set when the kernel determines that the last 256 pages requested by the process have all been found in the page cache (the value of consecutive cache hits is stored in the `ra->cache_hit` field). In this case, read-ahead is turned off because the kernel assumes that all the pages required by the process are already in the cache.

When is the read-ahead algorithm executed? This happens in the following cases:

- When the kernel handles a User Mode request to read pages of file data; this event triggers the invocation of the `page_cache_readahead( )` function (see step 4c in the description of the `do_generic_file_read( )` function in the section "[Reading from a File](#)" earlier in this chapter).
- When the kernel allocates a page for a file memory mapping (see the `filemap_nopage( )` function in the section "[Demand Paging for Memory Mapping](#)" later in this chapter, which again invokes the `page_cache_readahead( )` function).
- When a User Mode application executes the `readahead( )` system call, which explicitly triggers some read-ahead activity on a file descriptor.
- When a User Mode application executes the `posix_fadvise( )` system call with the `POSIX_FADV_NOREUSE` or `POSIX_FADV_WILLNEED` commands, which inform the kernel that a given range of file pages will be accessed in the near future.
- When a User Mode application executes the `madvice( )` system call with the `MADV_WILLNEED` command, which informs the kernel that a given range of pages in a file memory mapping region will be accessed in the near future.

16.1.2.1. The `page_cache_readahead( )` function

The `page_cache_readahead( )` function takes care of all read-ahead operations that are not explicitly triggered by ad-hoc system calls. It replenishes the current and ahead windows, updating their sizes according to the number of read-ahead hits, that is, according to how successful the read-ahead strategy was in the past accesses to the file.

The function is invoked when the kernel must satisfy a read request for one or more pages of a file, and acts on five parameters:

`mapping`

Pointer to the `address_space` object that describes the owner of the page

`ra`

Pointer to the `file_ra_state` descriptor of the file containing the page

`filp`

Address of the file object

`offset`

Offset of the page within the file

req_size

Number of pages yet to be read to complete the current read operation^[*]

[*] Actually, if the read operation involves a number of pages larger than the maximum size of the read-ahead window, the `page_cache_readahead()` function is invoked several times. Thus, the `req_size` parameter might be smaller than the number of pages yet to be read to complete the read operation.

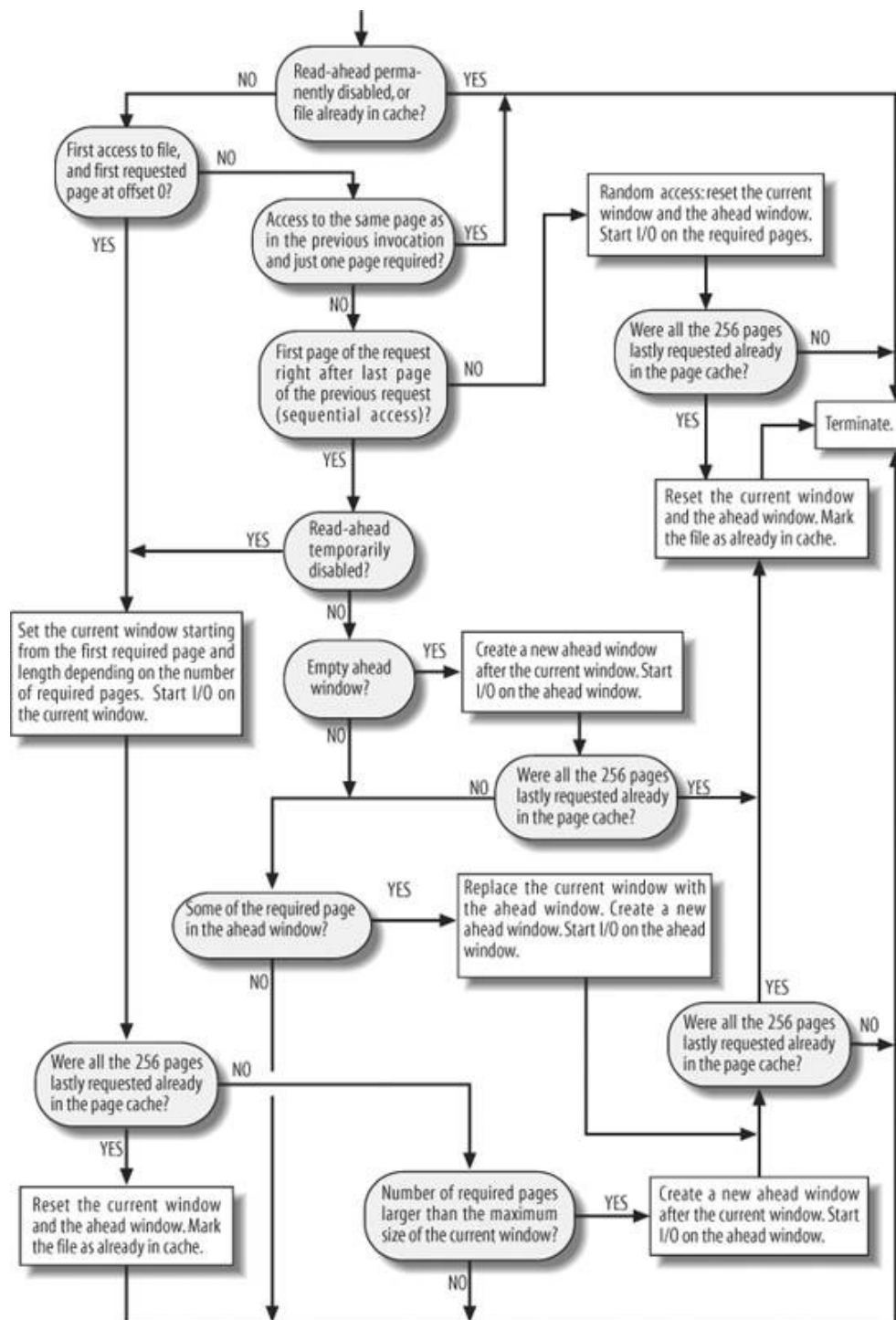
Figure 16-1 shows the flow diagram of `page_cache_readahead()`. The function essentially acts on the fields of the `file_ra_state` descriptor; thus, although the description of the actions in the flow diagram is quite informal, you can easily determine the actual steps performed by the function. For instance, in order to check whether the requested page is the same as the page previously read, the function checks whether the values of the `ra->prev_page` field and of the `offset` parameter coincide (see Table 16-3 earlier).

When the process accesses the file for the first time and the first requested page is the page at offset zero in the file, the function assumes that the process will perform sequential accesses. Thus, the function creates a new current window starting from the first page. The length of the initial current window is always a power of two, somewhat related to the number of pages requested by the process in the first read operation: the higher the number of requested pages, the larger the current window, up to the maximum value stored in the `ra->ra_pages` field. Conversely, when the process accesses the file for the first time but the first requested page is not at offset zero, the function assumes that the process will not perform sequential accesses. Thus, the function temporarily disables read-ahead (`ra->size` field is set to -1). However, a new current window is created when the function recognizes a sequential access while read-ahead is temporarily disabled.

If the ahead window does not already exist, it is created as soon as the function recognizes that the process has performed a sequential access in the current window. The ahead window always starts from the page following the last page of the current window. Its length, however, is related to the length of the current window as follows: if the `RA_FLAG_MISS` flag is set, the length of the ahead window is the length of the current window minus 2, or four pages if the result is less than four; otherwise, the length of the ahead window is either four times or two times the length of the current window. If the process continues to access the file in a sequential way, eventually the ahead window becomes the new current window, and a new ahead window is created. Thus, read-ahead is aggressively enhanced if the process reads the file sequentially.

As soon as the function recognizes a file access that is not sequential with respect to the previous one, the current and ahead windows are cleared (emptied) and the read-ahead is temporarily disabled. Read-ahead is restarted from scratch as soon as the process performs a read operation that is sequential with respect to the previous file access.

Figure 16-1. The flow diagram of the `page_cache_readahead()` function



Every time `page_cache_readahead()` creates a new window, it starts the read operations for the included pages. In order to read a chunk of pages, `page_cache_readahead()` invokes the `blockable_page_cache_readahead()` function. To reduce kernel overhead, the latter function adopts the following clever features:

- No reading is performed if the request queue that services the block device is read-congested (it does not make sense to increase congestion and block read-ahead).

- The page cache is checked against each page to be read; if the page is already in the page cache, it is simply skipped over.
- All the page frames needed by the read request are allocated at once before performing the read from disk. If not all page frames can be obtained, the read-ahead operation is performed only on the available pages. Again, there is little sense in deferring read-ahead until all page frames become available.
- Whenever possible, the read operations are submitted to the generic block layer by using multi-segment bio descriptors (see the section "[Segments](#)" in [Chapter 14](#)). This is done by the specialized `readpages` method of the `address_space` object, if defined; otherwise, it is done by repeatedly invoking the `readpage` method. The `readpage` method is described in the earlier section "[Reading from a File](#)" for the single-segment case only, but it is easy to adapt the description for the multi-segment case.

16.1.2.2. The `handle_ra_miss()` function

In some cases, the kernel must correct the read-ahead parameters, because the read-ahead strategy does not seem very effective. Let us consider the `do_generic_file_read()` function described in the section "[Reading from a File](#)" earlier in this chapter. The `page_cache_readahead()` function is invoked in step 4c. The flow diagram in [Figure 16-1](#) depicts two cases: either the requested page is in the current window or in the ahead window, hence it should have been read in advance, or it is not, and the function invokes `blockable_page_cache_readahead()` to read it. In both cases, `do_generic_file_read()` should find the page in the page cache in step 4d. If it is not found, this means that the page frame reclaiming algorithm has removed the page from the cache. In this case, `do_generic_file_read()` invokes the `handle_ra_miss()` function, which tunes the read-ahead algorithm by setting the `RA_FLAG_MISS` flag and by clearing the `RA_FLAG_INCACHE` flag.

16.1.3. Writing to a File

Recall that the `write()` system call involves moving data from the User Mode address space of the calling process into the kernel data structures, and then to disk. The `write` method of the file object permits each filesystem type to define a specialized write operation. In Linux 2.6, the `write` method of each disk-based filesystem is a procedure that basically identifies the disk blocks involved in the write operation, copies the data from the User Mode address space into some pages belonging to the page cache, and marks the buffers in those pages as dirty.

Many filesystems (including Ext2 or JFS) implement the `write` method of the file object by means of the `generic_file_write()` function, which acts on the following parameters:

`file`

File object pointer

`buf`

Address in the User Mode address space where the characters to be written into the file must be fetched

count

Number of characters to be written

ppos

Address of a variable storing the file offset from which writing must start

The function performs the following steps:

1. Initializes a local variable of type `iovec` containing the address and length of the User Mode buffer (see also the description of the `generic_file_read( )` function in the section "[Reading from a File](#)" earlier in this chapter).
2. Determines the address `inode` of the inode object that corresponds to the file to be written (`file->f_mapping->host`) and acquires the semaphore `inode->i_sem`. Thanks to this semaphore, only one process at a time can issue a `write( )` system call on the file.
3. Invokes the `init_sync_kiocb` macro to initialize a local variable of type `kiocb`. As explained in the section "[Reading from a File](#)" earlier in this chapter, the macro sets the `ki_key` field to `KIOCB_SYNC_KEY` (synchronous I/O operation), the `ki_filp` field to `file`, and the `ki_obj` field to `current`.
4. Invokes `__generic_file_aio_write_nolock( )` (see below) to mark the affected pages as dirty, passing the address of the local variables of type `iovec` and `kiocb`, the number of segments for the User Mode buffer only one in this case and the parameter `ppos`.
5. Releases the `inode->i_sem` semaphore.
6. Checks the `O_SYNC` flag of the file, the `S_SYNC` flag of the inode, and the `MS_SYNCHRONOUS` flag of the superblock; if at least one of them is set, it invokes the `sync_page_range( )` function to force the kernel to flush all pages in the page cache that have been touched in step 4, blocking the current process until the I/O data transfers terminate. In turn, `sync_page_range( )` executes either the `writepages` method of the `address_space` object, if defined, or the `mpage_writepages( )` function (see the section "[Writing Dirty Pages to Disk](#)" later in this chapter) to start the I/O transfers for the dirty pages; then, it invokes `generic_osync_inode( )` to flush to disk the inode and the associated buffers, and finally invokes `wait_on_page_bit( )` to suspend the current process until all `PG_writeback` bits of the flushed pages are cleared.
7. Returns the code returned by `__generic_file_aio_write_nolock( )`, usually the number of bytes effectively written.

The `__generic_file_aio_write_nolock( )` function receives four parameters: the address `kiocb` of a `kiocb` descriptor, the address `iov` of an array of `iovec` descriptors, the length of this array, and the address `ppos` of a variable that stores the file's current pointer. When invoked by `generic_file_write( )`, the array of `iovec` descriptors is composed of just one element describing the User Mode buffer that contains the data to be written.^[*]

[*] A variant of the `write( )` system call named `writev( )` allows an application to define multiple User Mode buffers from which the kernel fetches the data to be written on the file; the `generic_file_aio_write_nolock( )` function handles this case too. In the following pages, we will assume that the data will be fetched from just one User Mode buffer; however, guessing the additional steps to be performed when using multiple buffers is straightforward.

We now explain the actions of the `__generic_file_aio_write_nolock( )` function; for the sake of simplicity, we restrict the description to the most common case: a common mode operation raised by a `write( )` system call on a page-cached file. Later in this chapter we describe how this function behaves in other cases. As usual, we do not discuss how errors and anomalous conditions are handled.

The function executes the following steps:

1. Invokes `access_ok( )` to verify that the User Mode buffer described by the `iovec` descriptor is valid (the starting address and length have been received from the `sys_write( )` service routine, thus they must be checked before using them; see the section "[Verifying the Parameters](#)" in [Chapter 10](#)). If the parameters are not valid, it returns the `-EFAULT` error code.
2. Determines the address `inode` of the inode object that corresponds to the file to be written (`file->f_mapping->host`). Remember that if the file is a block device file, this is an inode in the `bdev` special filesystem (see [Chapter 14](#)).
3. Sets `current->backing_dev_info` to the address of the `backing_dev_info` descriptor of the file (`file->f_mapping->backing_dev_info`). Essentially, this setting allows the current process to write back the dirty pages owned by `file->f_mapping` even if the corresponding request queue is congested; see [Chapter 17](#).
4. If the `O_APPEND` flag of `file->flags` is on and the file is regular (not a block device file), it sets `*ppos` to the end of the file so that all new data is appended to it.
5. Performs several checks on the size of the file. For instance, the write operation must not enlarge a regular file so much as to exceed the per-user limit stored in `current->signal->rlim[RLIMIT_FSIZE]` (see the section "[Process Resource Limits](#)" in [Chapter 3](#)) and the filesystem limit stored in `inode->i_sb->s_maxbytes`. Moreover, if the file is not a "large file" (flag `O_LARGEFILE` of `file->f_flags` cleared), its size cannot exceed 2 GB. If any of these constraints is not enforced, it reduces the number of bytes to be written.
6. If set, it clears the `suid` flag of the file; also clears the `sgid` flag if the file is executable (see the section "[Access Rights and File Mode](#)" in [Chapter 1](#)). We don't want users to be able to modify setuid files.
7. Stores the current time of day in the `inode->mtime` field (the time of last file write operation) and in the `inode->ctime` field (the time of last inode change), and marks the inode object as dirty.
8. Starts a cycle to update all the pages of the file involved in the write operation. During each iteration, it performs the following substeps:
 - a. Invokes `find_lock_page( )` to search the page in the page cache (see the section "[Page Cache Handling Functions](#)" in [Chapter 15](#)). If this function finds the page, it increases its usage counter and sets its `PG_locked` flag.
 - b. If the page is not in the page cache, it allocates a new page frame and invokes `add_to_page_cache( )` to insert the page into the page cache; as explained in the section "[Page Cache Handling Functions](#)" in [Chapter 15](#), this function also increases the usage counter and sets the `PG_locked` flag. Moreover, the function inserts the new page into the inactive list of the memory zone (see [Chapter 17](#)).
 - c. Invokes the `prepare_write` method of the `address_space` object of the inode (`file->f_mapping`). The corresponding function takes care of allocating and initializing buffer heads for the page. We'll discuss in subsequent sections what this function does for regular files and block device files.
 - d. If the buffer is in high memory, it establishes a kernel mapping of the User Mode buffer (see the section "[Kernel Mappings of High-Memory Page Frames](#)" in [Chapter 8](#)). Then, it invokes `_copy_from_user( )` to copy the characters from the User Mode buffer to the page, and releases the kernel mapping.
 - e. Invokes the `commit_write` method of the `address_space` object of the inode (`file->f_mapping`). The corresponding function marks the underlying buffers as dirty so they are written to disk later. We discuss what this function does for regular files and block device files in the next two sections.
 - f. Invokes `unlock_page( )` to clear the `PG_locked` flag and wake up any process that is waiting for the page.
 - g. Invokes `mark_page_accessed( )` to update the page status for the memory reclaiming algorithm (see the section "[The Least Recently Used \(LRU\) Lists](#)" in [Chapter 17](#)).
 - h. Decreases the page usage counter to undo the increment in step 8a or 8b.
 - i. In this iteration, yet another page has been dirtied: it checks whether the ratio of dirty pages in the page cache has risen above a fixed threshold (usually, 40% of the pages in the system); if

- so, it invokes `writeback_inodes( )` to start flushing a few tens of pages to disk (see the section "[Looking for Dirty Pages To Be Flushed](#)" in [Chapter 15](#)).
- j. Invokes `cond_resched( )` to check the `TIF_NEED_RESCHED` flag of the current process and, if the flag is set, to invoke the `schedule( )` function.
- 9. Now all pages of the file involved in the write operation have been handled. Updates the value of `*ppos` to point right after the last character written.
- 10. Sets `current->backing_dev_info` to `NULL` (see step 3).
- 11. Terminates by returning the number of bytes effectively written.

16.1.3.1. The `prepare_write` and `commit_write` methods for regular files

The `prepare_write` and `commit_write` methods of the `address_space` object specialize the generic write operation implemented by `generic_file_write( )` for regular files and block device files. Both of them are invoked once for every page of the file that is affected by the write operation.

Each disk-based filesystem defines its own `prepare_write` method. As with read operations, this method is simply a wrapper for a common function. For instance, the Ext2 filesystem usually implements the `prepare_write` method by means of the following function:

```
int ext2_prepare_write(struct file *file, struct page *page,
                      unsigned from, unsigned to)
{
    return block_prepare_write(page, from, to, ext2_get_block);
}
```

The `ext2_get_block( )` function was already mentioned in the earlier section "[Reading from a File](#)"; it translates the block number relative to the file into a logical block number, which represents the position of the data on the physical block device.

The `block_prepare_write( )` function takes care of preparing the buffers and the buffer heads of the file's page by performing essentially the following steps:

1. Checks if the page is a buffer page (flag `PG_Private` set); if this flag is cleared, invokes `create_empty_buffers( )` to allocate buffer heads for all buffers included in the page (see the section "[Buffer Pages](#)" in [Chapter 15](#)).
2. For each buffer head relative to a buffer included in the page and affected by the write operation, the following is performed:
 - a. Resets the `BH_New` flag, if it is set (see below).
 - b. If the `BH_Mapped` flag is not set, the function performs the following substeps:
 1. Invokes the filesystem-dependent function whose address `get_block` was passed as a parameter. This function looks in the on-disk data structures of the filesystem and finds the logical block number of the buffer (relative to the beginning of the disk partition rather than the beginning of the regular file). The filesystem-dependent function stores this number in the `b_blocknr` field of the corresponding buffer head and sets its `BH_Mapped` flag. The `get_block` function could allocate a new physical block for the file (for instance, if the accessed block falls inside a "hole" of the regular file; see the section "[File Holes](#)" in [Chapter 18](#)). In this case, it sets the `BH_New` flag.

2. Checks the value of the `BH_New` flag; if it is set, invokes `unmap_underlying_metadata( )` to check whether some block device buffer page in the page cache includes a buffer referencing the same block on disk.^[*] This function essentially invokes `__find_get_block( )` to look up the old block in the page cache (see the section "[Searching Blocks in the Page Cache](#)" in [Chapter 15](#)). If such a block is found, the function clears its `BH_Dirty` flag and waits until any I/O data transfer on that buffer completes. Moreover, if the write operation does not rewrite the whole buffer in the page, it fills the unwritten portion with zero's. Then it considers the next buffer in the page.

[*] Although unlikely, this case might happen if a user writes blocks directly on the block device file, thus bypassing the filesystem.

- c. If the write operation does not rewrite the whole buffer and its `BH_Delay` and `BH_Uptodate` flags are not set (that is, the block has been allocated in the on-disk filesystem data structures and the buffer in RAM does not contain a valid image of the data), the function invokes `ll_rw_block( )` on the block to read its content from disk (see the section "[Submitting Buffer Heads to the Generic Block Layer](#)" in [Chapter 15](#)).
3. Blocks the current process until all read operations triggered in step 2c have been completed.
4. Returns 0.

Once the `prepare_write` method returns, the `generic_file_write( )` function updates the page with the data stored in the User Mode address space. Next, it invokes the `commit_write` method of the `address_space` object. This method is implemented by the `generic_commit_write( )` function for almost all disk-based non-journaling filesystems.

The `generic_commit_write( )` function performs essentially the following steps:

1. Invokes the `__block_commit_write( )` function. In turn, this function does the following:
 - a. Considers all buffers in the page that are affected by the write operation; for each of them, sets the `BH_Uptodate` and `BH_Dirty` flags of the corresponding buffer head.
 - b. Marks the corresponding inode as dirty. As seen in the section "[Looking for Dirty Pages To Be Flushed](#)" in [Chapter 15](#), this activity may require adding the inode to the list of dirty inodes of the superblock.
 - c. If all buffers in the buffer page are now up-to-date, it sets the `PG_uptodate` flag of the page.
 - d. Sets the `PG_dirty` flag of the page, and tags the page as dirty in its radix tree (see the section "[The Radix Tree](#)" in [Chapter 15](#)).
2. Checks whether the write operation enlarged the file. In this case, the function updates the `i_size` field of the file's inode.
3. Returns 0.

16.1.3.2. The `prepare_write` and `commit_write` methods for block device files

Write operations into block device files are very similar to the corresponding operations on regular files. In fact, the `prepare_write` method of the `address_space` object of block device files is usually implemented by the following function:

```
int blkdev_prepare_write(struct file *file, struct page *page,
                        unsigned from, unsigned to)
{
    return block_prepare_write(page, from, to, blkdev_get_block);
}
```

As you can see, the function is simply a wrapper to the `block_prepare_write( )` function already discussed in the previous section. The only difference, of course, is in the second parameter, which points to the function that must translate the file block number relative to the beginning of the file to a logical block number relative to the beginning of the block device. Remember that for block device files, the two numbers coincide. (See the earlier section ["Reading from a File"](#) for a discussion of the `blkdev_get_block( )` function.)

The `commit_write` method for block device files is implemented by the following simple wrapper function:

```
int blkdev_commit_write(struct file *file, struct page *page,
                      unsigned from, unsigned to)
{
    return block_commit_write(page, from, to);
}
```

As you can see, the `commit_write` method for block device files does essentially the same things as the `commit_write` method for regular files (we described the `block_commit_write( )` function in the previous section). The only difference is that the method does not check whether the write operation has enlarged the file; you simply cannot enlarge a block device file by appending characters to its last position.

16.1.4. Writing Dirty Pages to Disk

The net effect of the `write( )` system call consists of modifying the contents of some pages in the page cache optionally allocating the pages and adding them to the page cache if they were not already present. In some cases (for instance, if the file has been opened with the `O_SYNC` flag), the I/O data transfers start immediately (see step 6 of `generic_file_write( )` in the section ["Writing to a File"](#) earlier in this chapter). Usually, however, the I/O data transfer is delayed, as explained in the section ["Writing Dirty Pages to Disk"](#) in [Chapter 15](#).

When the kernel wants to effectively start the I/O data transfer, it ends up invoking the `writepages` method of the file's `address_space` object, which searches for dirty pages in the radix-tree and flushes them to disk. For instance, the Ext2 filesystem implements the `writepages` method by means of the following function:

```
int ext2_writepages(struct address_space *mapping,
                  struct writeback_control *wbc)
{
    return mpage_writepages(mapping, wbc, ext2_get_block);
}
```

As you can see, this function is a simple wrapper for the general-purpose `mpage_writepages( )` function; as a matter of fact, if a filesystem does not define the `writepages` method, the kernel invokes directly `mpage_writepages( )` passing `NULL` as third argument. The `ext2_get_block( )` function was already mentioned in the earlier section ["Reading from a File;"](#) it is the filesystem-dependent function that translates a file block number into a logical block number.

The `writeback_control` data structure is a descriptor that controls how the writeback operation has to be performed; we have already described it in the section "[Looking for Dirty Pages To Be Flushed](#)" in [Chapter 15](#).

The `mpage_writepages( )` function essentially performs the following actions:

1. If the request queue is write-congested and the process does not want to block, it returns without writing any page to disk.
2. Determines the file's initial page to be considered. If the `writeback_control` descriptor specifies the initial position in the file, the function translates it into a page index. Otherwise, if the `writeback_control` descriptor specifies that the process does not want to wait for the I/O data transfer to complete, it sets the initial page index to the value stored in `mapping->writeback_index` (that is, scanning begins from the last page considered in the previous writeback operation). Finally, if the process must wait until I/O data transfers complete, scanning starts from the first page of the file.
3. Invokes `find_get_pages_tag( )` to look up the descriptor of the dirty pages in the page cache (see the section "[The Tags of the Radix Tree](#)" in [Chapter 15](#)).
4. For each page descriptor retrieved in the previous step, the function performs the following steps:
 - a. Invokes `lock_page( )` to lock up the page.
 - b. Checks that the page is still valid and in the page cache (because another kernel control path could have acted upon the page between steps 3 and 4a).
 - c. Checks the `PG_writeback` flag of the page. If it is set, the page is already being flushed to disk. If the process must wait for the I/O data transfer to complete, it invokes `wait_on_page_bit( )` to block the current process until the `PG_writeback` flag is cleared; when this function terminates, any previously ongoing writeback operation is terminated. Otherwise, if the process does not want to wait, it checks the `PG_dirty` flag: if it is now cleared, the on-going writeback will take care of the page, thus unlocks it and jumps back to step 4a to continue with the next page.
 - d. If the `get_block` parameter is `NULL` (no `writepages` method defined), it invokes the `mapping->writepage` method of the `address_space` object of the file to flush the page to disk. Otherwise, if the `get_block` parameter is not `NULL`, it invokes the `mpage_writepage( )` function. See step 8 for details.
5. Invokes `cond_resched( )` to check the `TIF_NEED_RESCHED` flag of the current process and, if the flag is set, to invoke the `schedule( )` function.
6. If the function has not scanned all pages in the given range, or if the number of pages effectively written to disk is smaller than the value originally specified in the `writeback_control` descriptor, it jumps back to step 3.
7. If the `writeback_control` descriptor does not specify the initial position in the file, it sets the `mapping->writeback_index` field with the index of the last scanned page.
8. If the `mpage_writepage( )` function has been invoked in step 4d, and if that function returned the address of a bio descriptor, it invokes `mpage_bio_submit( )` (see below).

A typical filesystem such as Ext2 implements the `writepage` method as a wrapper for the general-purpose `block_write_full_page( )` function, passing to it the address of the filesystem-dependent `get_block` function. In turn, the `block_write_full_page( )` function is similar to `block_read_full_page( )` described in the section "[Reading from a File](#)" earlier in this chapter: it allocates buffer heads for the page (if the page was not already a buffer page), and invokes the `submit_bh( )` function on each of them, specifying the `WRITE` operation. As far as block device files are concerned, they implement the `writepage` method by using `blkdev_writepage( )`, which is a wrapper for `block_write_full_page( )`.

Many non-journaling filesystems rely on the `mpage_writepage( )` function rather than on the custom `writepage` method. This can improve performance because the `mpage_writepage( )` function tries to

submit the I/O transfers by collecting as many pages as possible in the same bio descriptor; in turn, this allows the block device drivers to exploit the scatter-gather DMA capabilities of the modern hard disk controllers.

To make a long story short, the `mpage_writepage( )` function checks whether the page to be written contains blocks that are not adjacent to disk, or whether the page includes a file hole, or whether some block on the page is not dirty or not up-to-date. If at least one of these conditions holds, the function falls back on the filesystem-dependent `writepage` method, as above. Otherwise, the function adds the page as a segment of a bio descriptor. The address of the bio descriptor is passed as parameter to the function; if it is `NULL`, `mpage_writepage( )` initializes a new bio descriptor and returns its address to the calling function, which in turn passes it back in the future invocations of `mpage_writepage( )`. In this way, several pages can be added to the same bio. If a page is not adjacent to the last added page in the bio, `mpage_writepage( )` invokes `mpage_bio_submit( )` to start the I/O data transfer on the bio, and allocates a new bio for the page.

The `mpage_bio_submit( )` function sets the `bi_end_io` method of the bio to the address of `mpage_end_io_write( )`, then invokes `submit_bio( )` to start the transfer (see the section ["Submitting Buffer Heads to the Generic Block Layer"](#) in [Chapter 15](#)). Once the data transfer successfully terminates, the completion function `mpage_end_io_write( )` wakes up any process waiting for the page transfer to complete, and destroys the bio descriptor.

16.2. Memory Mapping

As already mentioned in the section "[Memory Regions](#)" in [Chapter 9](#), a memory region can be associated with some portion of either a regular file in a disk-based filesystem or a block device file. This means that an access to a byte within a page of the memory region is translated by the kernel into an operation on the corresponding byte of the file. This technique is called memory mapping.

Two kinds of memory mapping exist:

Shared

Each write operation on the pages of the memory region changes the file on disk; moreover, if a process writes into a page of a shared memory mapping, the changes are visible to all other processes that map the same file.

Private

Meant to be used when the process creates the mapping just to read the file, not to write it. For this purpose, private mapping is more efficient than shared mapping. But each write operation on a privately mapped page will cause it to stop mapping the page in the file. Thus, a write does not change the file on disk, nor is the change visible to any other processes that access the same file. However, pages of a private memory mapping that have not been modified by the process are affected by file updates performed by other processes.

A process can create a new memory mapping by issuing an `mmap ( )` system call (see the section "[Creating a Memory Mapping](#)" later in this chapter). Programmers must specify either the `MAP_SHARED` flag or the `MAP_PRIVATE` flag as a parameter of the system call; as you can easily guess, in the former case the mapping is shared, while in the latter it is private. Once the mapping is created, the process can read the data stored in the file by simply reading from the memory locations of the new memory region. If the memory mapping is shared, the process can also modify the corresponding file by simply writing into the same memory locations. To destroy or shrink a memory mapping, the process may use the `munmap ( )` system call (see the later section "[Destroying a Memory Mapping](#)").

As a general rule, if a memory mapping is shared, the corresponding memory region has the `VM_SHARED` flag set; if it is private, the `VM_SHARED` flag is cleared. As we'll see later, an exception to this rule exists for read-only shared memory mappings.

16.2.1. Memory Mapping Data Structures

A memory mapping is represented by a combination of the following data structures :

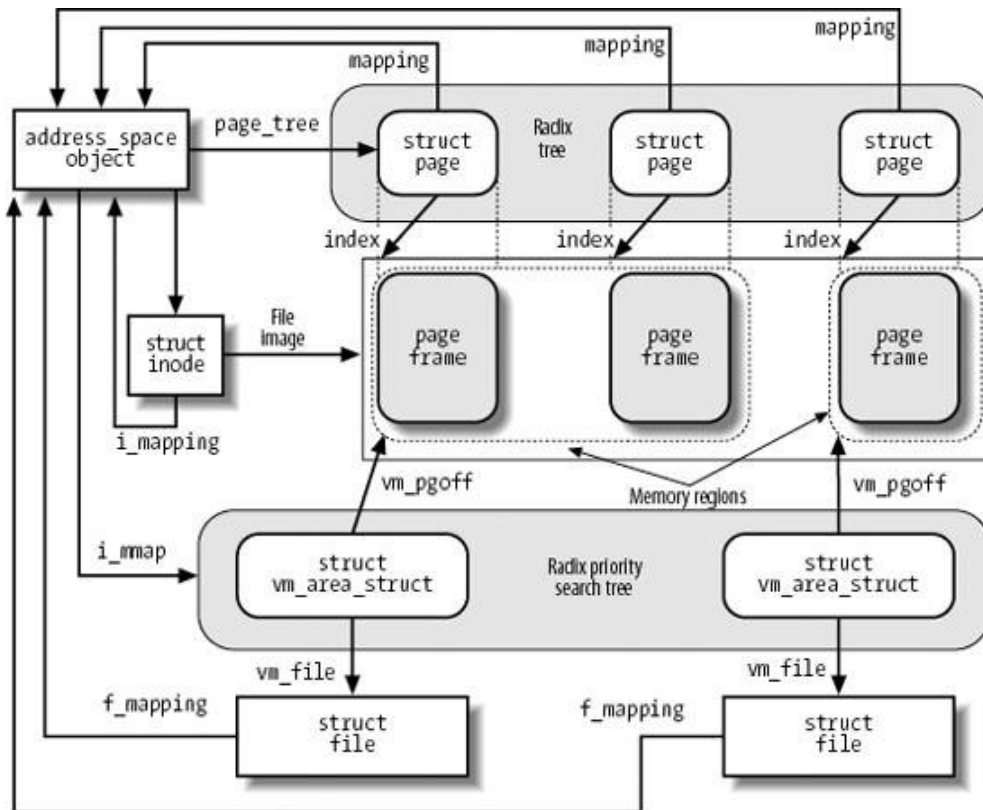
- The inode object associated with the mapped file
- The `address_space` object of the mapped file
- A file object for each different mapping performed on the file by different processes
- A `vm_area_struct` descriptor for each different mapping on the file

- A page descriptor for each page frame assigned to a memory region that maps the file

Figure 16-2 illustrates how the data structures are linked. On the left side of the image we show the inode, which identifies the file. The `i_mapping` field of each inode object points to the `address_space` object of the file. In turn, the `page_tree` field of each `address_space` object points to the radix tree of pages belonging to the address space (see the section "The Radix Tree" in Chapter 15), while the `i_mmap` field points to a second tree called the radix priority search tree (PST) of memory regions belonging to the address space. The main use of PST is for performing "reverse mapping," that is, for identifying quickly all processes that share a given page. We'll cover in detail PSTs in the next chapter, because they are used for page frame reclaiming. The link between file objects relative to the same file and the inode is established by means of the `f_mapping` field.

Each memory region descriptor has a `vm_file` field that links it to the file object of the mapped file (if that field is null, the memory region is not used in a memory mapping). The position of the first mapped location is stored into the `vm_pgoff` field of the memory region descriptor; it represents the file offset as a number of page-size units. The length of the mapped file portion is simply the length of the memory region, which can be computed from the `vm_start` and `vm_end` fields.

Figure 16-2. Data structures for file memory mapping



Pages of shared memory mappings are always included in the page cache; pages of private memory mappings are included in the page cache as long as they are unmodified. When a process tries to modify a page of a private memory mapping, the kernel duplicates the page frame and replaces the original page frame with the duplicate in the process Page Table; this is one of the applications of the Copy On Write mechanism that we discussed in Chapter 8. The original page frame still remains in the page cache, although it no longer belongs to the memory mapping since it is replaced by the duplicate. In turn, the duplicate is not inserted into the page

cache because it no longer contains valid data representing the file on disk.

Figure 16-2 also shows a few page descriptors of pages included in the page cache that refer to the memory-mapped file. Notice that the first memory region in the figure is three pages long, but only two page frames are allocated for it; presumably, the process owning the memory region has never accessed the third page.

The kernel offers several hooks to customize the memory mapping mechanism for every different filesystem. The core of memory mapping implementation is delegated to a file object's method named `mmap`. For most disk-based filesystems and for block device files, this method is implemented by a general function called `generic_file_mmap()`, which is described in the next section.

File memory mapping depends on the demand paging mechanism described in the section "Demand Paging" in Chapter 9. In fact, a newly established memory mapping is a memory region that doesn't include any page; as the process references an address inside the region, a Page Fault occurs and the Page Fault handler checks whether the `nopage` method of the memory region is defined. If `nopage` is not defined, the memory region doesn't map a file on disk; otherwise, it does, and the method takes care of reading the page by accessing the block device. Almost all disk-based filesystems and block device files implement the `nopage` method by means of the `filemap_nopage()` function.

16.2.2. Creating a Memory Mapping

To create a new memory mapping, a process issues an `mmap()` system call, passing the following parameters to it:

- A file descriptor identifying the file to be mapped.
- An offset inside the file specifying the first character of the file portion to be mapped.
- The length of the file portion to be mapped.
- A set of flags. The process must explicitly set either the `MAP_SHARED` flag or the `MAP_PRIVATE` flag to specify the kind of memory mapping requested.^[*]

[*] The process could also set the `MAP_ANONYMOUS` flag to specify that the new memory region is anonymous that is, not associated with any disk-based file (see the section "Demand Paging" in Chapter 9). A process can also create a memory region that is both `MAP_SHARED` and `MAP_ANONYMOUS`: in this case, the region maps a special file in the `tmpfs` filesystem (see the section "IPC Shared Memory" in Chapter 19), which can be accessed by all the process's descendants.

- A set of permissions specifying one or more types of access to the memory region: read access (`PROT_READ`), write access (`PROT_WRITE`), or execution access (`PROT_EXEC`).
- An optional linear address, which is taken by the kernel as a hint of where the new memory region should start. If the `MAP_FIXED` flag is specified and the kernel cannot allocate the new memory region starting from the specified linear address, the system call fails.

The `mmap()` system call returns the linear address of the first location in the new memory region. For compatibility reasons, in the 80 x 86 architecture, the kernel reserves two entries in the system call table for `mmap()`: one at index 90 and the other at index 192. The former entry corresponds to the `old_mmap()` service routine (used by older C libraries), while the latter one corresponds to the `sys_mmap2()` service routine (used by recent C libraries). The two service routines differ only in how the six parameters of the system call are passed. Both of them end up invoking the `do_mmap_pgoff()` function described in the section "Allocating a Linear Address Interval" in Chapter 9. We now complete that description by detailing the steps performed only when creating a memory region that maps a file. We thus describe the case where the

`file` parameter (pointer to a file object) of `do_mmap_pgoff( )` is non-null. For the sake of clarity, we refer to the enumeration used to describe `do_mmap_pgoff( )` and point out the additional steps performed under the new condition.

Step 1

Checks whether the `mmap` file operation for the file to be mapped is defined; if not, it returns an error code. A `NULL` value for `mmap` in the file operation table indicates that the corresponding file cannot be mapped (for instance, because it is a directory).

Step 2

The `get_unmapped_area( )` function invokes the `get_unmapped_area` method of the file object, if it is defined, so as to allocate an interval of linear addresses suitable for the memory mapping of the file. The disk-based filesystems do not define this method; in this case, as explained in the section "[Memory Region Handling](#)" in [Chapter 9](#), the `get_unmapped_area( )` function ends up invoking the `get_unmapped_area` method of the memory descriptor.

Step 3

In addition to the usual consistency checks, it compares the kind of memory mapping requested (stored in the `flags` parameter of the `mmap( )` system call) and the flags specified when the file was opened (stored in the `file->f_mode` field). In particular:

- ◊ If a shared writable memory mapping is required, it checks that the file was opened for writing and that it was not opened in append mode (`O_APPEND` flag of the `open( )` system call).
- ◊ If a shared memory mapping is required, it checks that there is no mandatory lock on the file (see the section "[File Locking](#)" in [Chapter 12](#)).
- ◊ For every kind of memory mapping, it checks that the file was opened for reading.

If any of these conditions is not fulfilled, an error code is returned.

Moreover, when initializing the value of the `vm_flags` field of the new memory region descriptor, it sets the `VM_READ`, `VM_WRITE`, `VM_EXEC`, `VM_SHARED`, `VM_MAYREAD`, `VM_MAYWRITE`, `VM_MAYEXEC`, and `VM_MAYSHARE` flags according to the access rights of the file and the kind of requested memory mapping (see the section "[Memory Region Access Rights](#)" in [Chapter 9](#)). As an optimization, the `VM_SHARED` and `VM_MAYWRITE` flags are cleared for nonwritable shared memory mapping. This can be done because the process is not allowed to write into the pages of the memory region, so the mapping is treated the same as a private mapping; however, the kernel actually allows other processes that share the file to read the pages in this memory region.

Step 10

Initializes the `vm_file` field of the memory region descriptor with the address of the file object and increases the file's usage counter. Invokes the `mmap` method for the file being mapped, passing as parameters the address of the file object and the address of the memory region descriptor. For most filesystems, this method is implemented by the `generic_file_mmap( )` function, which performs the following operations:

- a. Stores the current time in the `i_atime` field of the file's inode and marks the inode as dirty.
- b. Initializes the `vm_ops` field of the memory region descriptor with the address of the `generic_file_vm_ops` table. All methods in this table are null, except the `nopage` method, which is implemented by the `filemap_nopage( )` function, and the `populate`

method, which is implemented by the `filemap_populate( )` function (see "[Non-Linear Memory Mappings](#)" later in this chapter).

Step 11

Increases the `i_writecount` field of the file's inode, that is, the usage counter for writing processes.

16.2.3. Destroying a Memory Mapping

When a process is ready to destroy a memory mapping, it invokes `munmap( )`; this system call can also be used to reduce the size of each kind of memory region. The parameters used are:

- The address of the first location in the linear address interval to be removed.
- The length of the linear address interval to be removed.

The `sys_munmap( )` service routine of the system call essentially invokes the `do_munmap( )` function already described in the section "[Releasing a Linear Address Interval](#)" in [Chapter 9](#). Notice that there is no need to flush to disk the contents of the pages included in a writable shared memory mapping to be destroyed. In fact, these pages continue to act as a disk cache because they are still included in the page cache.

16.2.4. Demand Paging for Memory Mapping

For reasons of efficiency, page frames are not assigned to a memory mapping right after it has been created, but at the last possible moment—that is, when the process attempts to address one of its pages, thus causing a Page Fault exception.

We saw in the section "[Page Fault Exception Handler](#)" in [Chapter 9](#) how the kernel verifies whether the faulty address is included in some memory region of the process; if so, the kernel checks the Page Table entry corresponding to the faulty address and invokes the `do_no_page( )` function if the entry is null (see the section "[Demand Paging](#)" in [Chapter 9](#)).

The `do_no_page( )` function performs all the operations that are common to all types of demand paging, such as allocating a page frame and updating the Page Tables. It also checks whether the `nopage` method of the memory region involved is defined. In the section "[Demand Paging](#)" in [Chapter 9](#), we described the case in which the method is undefined (anonymous memory region); now we complete the description by discussing the main actions performed by the function when the method is defined:

1. Invokes the `nopage` method, which returns the address of a page frame that contains the requested page.
2. If the process is trying to write into the page and the memory mapping is private, it avoids a future Copy On Write fault by making a copy of the page just read and inserting it into the inactive list of pages (see [Chapter 17](#)). If the private memory mapping region does not already have a slave anonymous memory region that includes the new page, it either adds a new slave anonymous memory region or extends an existing one (see the section "[Memory Regions](#)" in [Chapter 9](#)). In the following steps, the function uses the new page instead of the page returned by the `nopage` method, so that the latter is not modified by the User Mode process.
3. If some other process has truncated or invalidated the page (the `truncate_count` field of the

`address_space` descriptor is used for this kind of check), the function retries getting the page by jumping back to step 1.

4. Increases the `rss` field of the process memory descriptor to indicate that a new page frame has been assigned to the process.
5. Sets up the Page Table entry corresponding to the faulty address with the address of the page frame and the page access rights included in the memory region `vm_page_prot` field.
6. If the process is trying to write into the page, it forces the `Read/Write` and `Dirty` bits of the Page Table entry to 1. In this case, either the page frame is exclusively assigned to the process, or the page is shared; in both cases, writing to it should be allowed.

The core of the demand paging algorithm consists of the memory region's `nopage` method. Generally speaking, it must return the address of a page frame that contains the page accessed by the process. Its implementation depends on the kind of memory region in which the page is included.

When handling memory regions that map files on disk, the `nopage` method must first search for the requested page in the page cache. If the page is not found, the method must read it from disk. Most filesystems implement the `nopage` method by means of the `filemap_nopage( )` function, which receives three parameters:

`area`

Descriptor address of the memory region, including the required page

`address`

Linear address of the required page

`type`

Pointer to a variable in which the function writes the type of page fault detected by the function (`VM_FAULT_MAJOR` or `VM_FAULT_MINOR`)

The `filemap_nopage( )` function executes the following steps:

1. Gets the file object address `file` from the `area->vm_file` field. Derives the `address_space` object address from `file->f_mapping`. Derives the inode object address from the `host` field of the `address_space` object.
2. Uses the `vm_start` and `vm_pgoff` fields of `area` to determine the offset within the file of the data corresponding to the page starting from `address`.
3. Checks whether the file offset exceeds the file size. When this happens, it returns `NULL`, which means failure in allocating the new page, unless the Page Fault was caused by a debugger tracing another process through the `ptrace( )` system call. We are not going to discuss this special case.
4. If the `VM_RANDOM_READ` flag of the memory region is set (see below), we may assume that the process is reading the pages of the memory mapping in a random way. In this case, it ignores read-ahead by jumping to step 10.
5. If the `VM_SEQ_READ` flag of the memory region is set (see below), we may assume that the process is reading the pages of the memory mapping in a strictly sequential way. In this case, it invokes `page_cache_readahead( )` to perform read-ahead starting from the faulty page (see the section ["Read-Ahead of Files"](#) earlier in this chapter).
6. Invokes `find_get_page( )` to look in the page cache for the page identified by the `address_space` object and the file offset. If the page is found, it jumps to step 11.
7. If the function has reached this point, the page has not been found in the page cache. Checks the `VM_SEQ_READ` flag of the memory region:

- ◆ If the flag is set, the kernel is aggressively reading in advance the pages of the memory region, hence the read-ahead algorithm has failed: it invokes `handle_ra_miss( )` to tune up the read-ahead parameters (see the section "Read-Ahead of Files" earlier in this chapter), then jumps to step 10.
 - ◆ Otherwise, if the flag is clear, it increases by one the `mmap_miss` counter in the `file_ra_state` descriptor of the file. If the number of misses is much larger than the number of hits (stored in the `mmap_hit` counter), it ignores read-ahead by jumping to step 10.
8. If read-ahead is not permanently disabled (`ra_pages` field in the `file_ra_state` descriptor greater than zero), it invokes `do_page_cache_readahead( )` to read a set of pages surrounding the requested page.
 9. Invokes `find_get_page( )` to check whether the requested page is in the page cache; if it is there, jumps to step 11.
 10. Invokes `page_cache_read( )`. This function checks whether the requested page is already in the page cache and, if it is not there, allocates a new page frame, adds it to the page cache, and executes the `mapping->a_ops->readpage` method to schedule an I/O operation that reads the page's contents from disk.
 11. Invokes the `grab_swap_token( )` function to possibly assign the swap token to the current process (see the section "The Swap Token" in Chapter 17).
 12. The requested page is now in the page cache. Increases by one the `mmap_hit` counter of the `file_ra_state` descriptor of the file.
 13. If the page is not up-to-date (`PG_uptodate` flag clear), it invokes `lock_page( )` to lock up the page, executes the `mapping->a_ops->readpage` method to trigger the I/O data transfer, and invokes `wait_on_page_bit( )` to sleep until the page is unlocked—that is, until the data transfer completes.
 14. Invokes `mark_page_accessed( )` to mark the requested page as accessed (see next chapter).
 15. If an up-to-date version of the page was found in the page cache, it sets `*type` to `VM_FAULT_MINOR`; otherwise sets it to `VM_FAULT_MAJOR`.
 16. Returns the address of the requested page.

A User Mode process can tailor the read-ahead behavior of the `filemap_nopage( )` function by using the `madvise( )` system call. The `MADV_RANDOM` command sets the `VM_RANDOM_READ` flag of the memory region to specify that the pages of the memory region will be accessed in random order; the `MADV_SEQUENTIAL` command sets the `VM_SEQ_READ` flag to specify that the pages will be accessed in strictly sequential order; finally, the `MADV_NORMAL` command resets both the `VM_RANDOM_READ` and `VM_SEQ_READ` flags to specify that the pages will be accessed in an unspecified order.

16.2.5. Flushing Dirty Memory Mapping Pages to Disk

The `msync( )` system call can be used by a process to flush to disk dirty pages belonging to a shared memory mapping. It receives as its parameters the starting address of an interval of linear addresses, the length of the interval, and a set of flags that have the following meanings:

MS_SYNC

Asks the system call to suspend the process until the I/O operation completes. In this way, the calling process can assume that when the system call terminates, all pages of its memory mapping have been flushed to disk.

MS_ASYNC (complement of **MS_SYNC**)

Asks the system call to return immediately without suspending the calling process.

MS_INVALIDATE

Asks the system call to invalidate other memory mappings of the same file (not really implemented, because useless in Linux).

The `sys_msync( )` service routine invokes `msync_interval( )` on each memory region included in the interval of linear addresses. In turn, the latter function performs the following operations:

1. If the `vm_file` field of the memory region descriptor is `NULL`, or if the `VM_SHARED` flag is clear, it returns 0 (the memory region is not a writable shared memory mapping of a file).
2. Invokes the `filemap_sync( )` function, which scans the Page Table entries corresponding to the linear address intervals included in the memory region. For each page found, it resets the `Dirty` flag in the corresponding page table entry and invokes `flush_tlb_page( )` to flush the corresponding translation lookaside buffers; then, it sets the `PG_dirty` flag in the page descriptor to mark the page as dirty.
3. If the `MS_ASYNC` flag is set, it returns. Therefore, the practical effect of the `MS_ASYNC` flag consists of setting the `PG_dirty` flags of the pages in the memory region; the system call does not actually start the I/O data transfers.
4. If the function has reached this point, the `MS_SYNC` flag is set, hence the function must flush the pages in the memory region to disk and put the current process to sleep until all I/O data transfers terminate. In order to do this, the function acquires the `i_sem` semaphore of the file's inode.
5. Invokes the `filemap_fdatawrite( )` function, which receives the address of the file's `address_space` object. This function essentially sets up a `writeback_control` descriptor with the `WB_SYNC_ALL` synchronization mode, and checks whether the address space has a built-in `writepages` method. If so, it invokes the corresponding function and returns. In the opposite case, it executes the `mpage_writepages( )` function. (See the section "[Writing Dirty Pages to Disk](#)" earlier in this chapter.)
6. Checks whether the `fsync` method of the file object is defined; if so, executes it. For regular files, this method usually limits itself to flushing the inode object of the file to disk. For block device files, however, the method invokes `sync_blockdev( )`, which activates the I/O data transfer of all dirty buffers of the device.
7. Executes the `filemap_fdatawait( )` function. We recall from the section "[The Tags of the Radix Tree](#)" in [Chapter 15](#) that a radix tree in the page cache identifies all pages that are currently being written to disk by means of the `PAGECACHE_TAG_WRITEBACK` tag. The function quickly scans the portion of the radix tree that covers the given interval of linear addresses looking for pages having the `PG_writeback` flag set; for each such page, the function invokes `wait_on_page_bit( )` to sleep until the `PG_writeback` flag is cleared that is, until the ongoing I/O data transfer on the page terminates.
8. Releases the `i_sem` semaphore of the file and returns.

16.2.6. Non-Linear Memory Mappings

The Linux 2.6 kernel offers yet another kind of access method for regular files: the non-linear memory mappings. Basically, a non-linear memory mapping is a file memory mapping as described previously, but its memory pages are not mapped to sequential pages on the file; rather, each memory page maps a random (arbitrary) page of file's data.

Of course, a User Mode application might achieve the same result by invoking the `mmap( )` system call repeatedly, each time on a different 4096-byte-long portion of the file. However, this approach is not very

efficient for non-linear mapping of large files, because each mapping page requires its own memory region.

In order to support non-linear memory mapping, the kernel makes use of a few additional data structures. First of all, the `VM_NONLINEAR` flag of the memory region descriptor specifies that the memory region contains a non-linear mapping. All descriptors of non-linear mapping memory regions for a given file are collected in a doubly linked circular list rooted at the `i_mmap_nonlinear` field of the `address_space` object.

To create a non-linear memory mapping, the User Mode application first creates a normal shared memory mapping with the `mmap( )` system call. Then, the application remaps some of the pages in the memory mapping region by invoking `remap_file_pages( )`. The `sys_remap_file_pages( )` service routine of the system call receives four parameters:

<code>start</code>	A linear address inside a shared file memory mapping region of the calling process
<code>size</code>	Size of the remapped portion of the file in bytes
<code>prot</code>	Unused (must be zero)
<code>pgoff</code>	Page index of the initial file's page to be remapped
<code>flags</code>	Flags controlling the non-linear memory mapping

The service routine remaps the portion of the file's data identified by the `pgoff` and `size` parameters starting from the `start` linear address. If either the memory region is not shared or it is not large enough to include all the pages requested for the mapping, the system call fails and an error code is returned. Essentially, the service routine inserts the memory region in the `i_mmap_nonlinear` list of the file and invokes the `populate` method of the memory region.

For all regular files, the `populate` method is implemented by the `filemap_populate( )` function, which executes the following steps:

1. Checks whether the `MAP_NONBLOCK` flag in the `flags` parameter of the `remap_file_pages( )` system call is clear; if so, it invokes `do_page_cache_readahead( )` to read in advance the pages of the file to be remapped.
2. For each page to be remapped, performs the following substeps:
 - a. Checks whether the page descriptor is already included in the page cache; if it is not there and the `MAP_NONBLOCK` flag is cleared, it reads the page from disk.
 - b. If the page descriptor is in the page cache, it updates the Page Table entry of the corresponding linear address so that it points to the page frame, and updates the counter of pages in the memory region descriptor.
 - c. Otherwise, if the page descriptor has not been found in the page cache, it stores the offset of the file's page in the 32 highest-order bits of the Page Table entry for the corresponding linear

address; also, clears the `Present` bit of the Page Table entry and sets the `Dirty` bit.

As explained in the section "[Demand Paging](#)" in [Chapter 9](#), when handling a demand-paging fault the `handle_pte_fault()` function checks the `Present` and `Dirty` bits in the Page Table entry; if they have the values corresponding to a non-linear memory mapping, `handle_pte_fault()` invokes the `do_file_page()` function, which extracts the index of the requested file's page from the high-order bits of the Page Table entry; then, `do_file_page()` invokes the `populate` method of the memory region to read the page from disk and update the Page Table entry itself.

Because the memory pages of a non-linear memory mapping are included in the page cache according to the page index relative to the beginning of the file rather than the index relative to the beginning of the memory region, non-linear memory mappings are flushed to disk exactly like linear memory mappings (see the section "[Flushing Dirty Memory Mapping Pages to Disk](#)" earlier in this chapter).

16.3. Direct I/O Transfers

As we have seen, in Version 2.6 of Linux, there is no substantial difference between accessing a regular file through the filesystem, accessing it by referencing its blocks on the underlying block device file, or even establishing a file memory mapping. There are, however, some highly sophisticated programs (self-caching applications) that would like to have full control of the whole I/O data transfer mechanism. Consider, for example, high-performance database servers: most of them implement their own caching mechanisms that exploit the peculiar nature of the queries to the database. For these kinds of programs, the kernel page cache doesn't help; on the contrary, it is detrimental for the following reasons:

- Lots of page frames are wasted to duplicate disk data already in RAM (in the user-level disk cache).
- The `read( )` and `write( )` system calls are slowed down by the redundant instructions that handle the page cache and the read-ahead; ditto for the paging operations related to the file memory mappings.
- Rather than transferring the data directly between the disk and the user memory, the `read( )` and `write( )` system calls make two transfers: between the disk and a kernel buffer and between the kernel buffer and the user memory.

Because block hardware devices must be handled through interrupts and Direct Memory Access (DMA), and this can be done only in Kernel Mode, some sort of kernel support is definitely required to implement self-caching applications.

Linux offers a simple way to bypass the page cache: direct I/O transfers. In each I/O direct transfer, the kernel programs the disk controller to transfer the data directly from/to pages belonging to the User Mode address space of a self-caching application.

As we know, each data transfer proceeds asynchronously. While it is in progress, the kernel may switch the current process, the CPU may return to User Mode, the pages of the process that raised the data transfer might be swapped out, and so on. This works just fine for ordinary I/O data transfers because they involve pages of the disk caches. Disk caches are owned by the kernel, cannot be swapped out, and are visible to all processes in Kernel Mode.

On the other hand, direct I/O transfers should move data within pages that belong to the User Mode address space of a given process. The kernel must take care that these pages are accessible by every process in Kernel Mode and that they are not swapped out while the data transfer is in progress. Let us see how this is achieved.

When a self-caching application wishes to directly access a file, it opens the file specifying the `O_DIRECT` flag (see the section "[The `open\( \)` System Call](#)" in [Chapter 12](#)). While servicing the `open( )` system call, the `dentry_open( )` function checks whether the `direct_IO` method is implemented for the `address_space` object of the file being opened, and returns an error code in the opposite case. The `O_DIRECT` flag can also be set for a file already opened by using the `F_SETFL` command of the `fcntl( )` system call.

Let us consider first the case where the self-caching application issues a `read( )` system call on a file with `O_DIRECT`. As mentioned in the section "[Reading from a File](#)" earlier in this chapter, the `read` file method is usually implemented by the `generic_file_read( )` function, which initializes the `iovec` and `kiocb` descriptors and invokes `__generic_file_aio_read( )`. The latter function verifies that the User Mode buffer described by the `iovec` descriptor is valid, then checks whether the `O_DIRECT` flag of the file is set. When invoked by a `read( )` system call, the function executes a code fragment essentially equivalent to the following:

```

if (filp->f_flags & O_DIRECT) {
    if (count == 0 || *ppos > filp->f_mapping->host->i_size)
        return 0;
    retval = generic_file_direct_IO(READ, iocb, iov, *ppos, 1);
    if (retval > 0)
        *ppos += retval;
    file_accessed(filp);
    return retval;
}

```

The function checks the current values of the file pointer, the file size, and the number of requested characters, and then invokes the `generic_file_direct_IO( )` function, passing to it the `READ` operation type, the `iocb` descriptor, the `iovec` descriptor, the current value of the file pointer, and the number of User Mode buffers specified in the `io_vec` descriptor (one). When `generic_file_direct_IO( )` terminates, `_generic_file_aio_read( )` updates the file pointer, sets the access timestamp on the file's inode, and returns.

Something similar happens when a `write( )` system call is issued on a file having the `O_DIRECT` flag set. As mentioned in the section "[Writing to a File](#)" earlier in this chapter, the `write` method of the file ends up invoking `generic_file_aio_write_nolock( )`: this function checks whether the `O_DIRECT` flag is set and, if so, invokes the `generic_file_direct_IO( )` function, this time specifying the `WRITE` operation type.

The `generic_file_direct_IO( )` function acts on the following parameters:

<code>rw</code>	Type of operation: <code>READ</code> or <code>WRITE</code>
<code>iocb</code>	Pointer to a <code>kiocb</code> descriptor (see Table 16-1)
<code>iov</code>	Pointer to an array of <code>iovec</code> descriptors (see the section " Reading from a File " earlier in this chapter)
<code>offset</code>	File offset
<code>nr_segs</code>	Number of <code>iovec</code> descriptors in the <code>iov</code> array

The steps performed by `generic_file_direct_IO( )` are the following:

1. Gets the address `file` of the file object from the `ki_filp` field of the `kiocb` descriptor, and the address mapping of the `address_space` object from the `file->f_mapping` field.
2. If the type of operation is `WRITE` and if one or more processes have created a memory mapping associated with a portion of the file, it invokes `unmap_mapping_range( )` to unmap all pages of the file. This function also ensures that if any Page Table entry corresponding to a page to be unmapped has the `Dirty` bit set, then the corresponding page is marked as dirty in the page cache.

3. If the radix tree rooted at `mapping` is not empty (`mapping->numpages` greater than zero), it invokes the `filemap_fdatawrite( )` and `filemap_fdatawait( )` functions to flush all dirty pages to disk and to wait until the I/O operations complete (see the section "[Flushing Dirty Memory Mapping Pages to Disk](#)" earlier in this chapter). (Even if the self-caching application is accessing the file directly, there could be other applications in the system that access the file through the page cache. To avoid data loss, the disk image is synchronized with the page cache before starting the direct I/O transfer.)
4. Invokes the `direct_IO` method of the `mapping` address space (see the following paragraphs).
5. If the operation type was `WRITE`, it invokes `invalidate_inode_pages2( )` to scan all pages in the radix tree of `mapping` and to release them. The function also clears the User Mode Page Table entries that refer to those pages.

In most cases, the `direct_IO` method is a wrapper for the `__blockdev_direct_IO( )` function. This function is quite complex and invokes a large number of auxiliary data structures and functions; however, it executes essentially the same kind of operations already described in this chapter: it splits the data to be read or written in suitable blocks, locates the data on disk, and fills up one or more bio descriptors that describe the I/O operations to be performed. Of course, the data will be read or written directly in the User Mode buffers specified by the `iovec` descriptors in the `iov` array. The bio descriptors are submitted to the generic block layer by invoking the `submit_bio( )` function (see the section "[Submitting Buffer Heads to the Generic Block Layer](#)" in [Chapter 15](#)). Usually, the `__blockdev_direct_IO( )` function does not return until all direct I/O transfers have been completed; thus, once the `read( )` or `write( )` system call returns, the self-caching application can safely access the buffers containing the file data.

16.4. Asynchronous I/O

The POSIX 1003.1 standard defines a set of library functions listed in [Table 16-4](#) for accessing the files in an asynchronous way. "Asynchronous" essentially means that when a User Mode process invokes a library function to read or write a file, the function terminates as soon as the read or write operation has been enqueued, possibly even before the actual I/O data transfer takes place. The calling process can thus continue its execution while the data is being transferred.

Table 16-4. The POSIX library functions for asynchronous I/O

Function	Description
<code>aio_read()</code>	Asynchronously reads some data from a file
<code>aio_write()</code>	Asynchronously writes some data into a file
<code>aio_fsync()</code>	Requests a flush operation for all outstanding asynchronous I/O operations (does not block)
<code>aio_error()</code>	Gets the error code for an outstanding asynchronous I/O operation
<code>aio_return()</code>	Gets the return code for a completed asynchronous I/O operation
<code>aio_cancel()</code>	Cancels an outstanding asynchronous I/O operation
<code>aio_suspend()</code>	Suspends the process until at least one of several outstanding I/O operations completes

Using asynchronous I/O is quite simple. The application opens the file by means of the usual `open( )` system call. Then, it fills up a control block of type `struct aiocb` with the information describing the requested operation. The most commonly used fields of the `struct aiocb` control block are:

- `aio_fildes`
The file descriptor of the file (as returned by the `open( )` system call)
- `aio_buf`
The User Mode buffer for the file's data

`aio_nbytes`

How many bytes should be transferred

`aio_offset`

Position in the file where the read or write operation will start (it is independent of the "synchronous" file pointer)

Finally, the application passes the address of the control block to either `aio_read( )` or `aio_write( )`; both functions terminate as soon as the requested I/O data transfer has been enqueued by the system library or kernel. The application can later check the status of the outstanding I/O operation by invoking `aio_error( )`, which returns `EINPROGRESS` if the data transfer is still in progress, 0 if it is successfully completed, or an error code in case of failure. The `aio_return( )` function returns the number of bytes effectively read or written by a completed asynchronous I/O operation, or -1 in case of failure.

16.4.1. Asynchronous I/O in Linux 2.6

Asynchronous I/O can be implemented by a system library without any kernel support at all. Essentially, the `aio_read( )` or `aio_write( )` library function clones the current process and lets the child invoke the synchronous `read( )` or `write( )` system calls; then, the parent terminates the `aio_read( )` or `aio_write( )` function and continues the execution of the program, hence it does not wait for the synchronous operation started by the child to finish. However, this "poor man's" version of the POSIX functions is significantly slower than a version that uses a kernel-level implementation of asynchronous I/O.

The Linux 2.6 kernel version sports a set of system calls for asynchronous I/O. However, in Linux 2.6.11 this feature is a work in progress, and asynchronous I/O works properly only for files opened with the `O_DIRECT` flag set (see the previous section). The system calls for asynchronous I/O are listed in [Table 16-5](#).

Table 16-5. Linux system calls for asynchronous I/O

System call

Description

`io_setup( )`

Initializes an asynchronous context for the current process

`io_submit( )`

Submits one or more asynchronous I/O operations

`io_getevents( )`

Gets the completion status of some outstanding asynchronous I/O operations

`io_cancel( )`

Cancels an outstanding I/O operation

`io_destroy( )`

Removes an asynchronous context for the current process

16.4.1.1. The asynchronous I/O context

If a User Mode process wants to make use of the `io_submit( )` system call to start an asynchronous I/O operation, it must create beforehand an asynchronous I/O context.

Basically, an asynchronous I/O context (in short, AIO context) is a set of data structures that keep track of the on-going progresses of the asynchronous I/O operations requested by the process. Each AIO context is associated with a `kioctx` object, which stores all information relevant for the context. An application might create several AIO contexts; all `kioctx` descriptors of a given process are collected in a singly linked list rooted at the `ioctx_list` field of the memory descriptor (see [Table 9-2](#) in [Chapter 9](#)).

We are not going to discuss in detail the `kioctx` object; however, we should pinpoint an important data structure referenced by the `kioctx` object: the AIO ring.

The AIO ring is a memory buffer in the address space of the User Mode process that is also accessible by all processes in Kernel Mode. The User Mode starting address and length of the AIO ring are stored in the `ring_info.mmap_base` and `ring_info.mmap_size` fields of the `kioctx` object, respectively. The descriptors of all page frames composing the AIO ring are stored in an array pointed to by the `ring_info.ring_pages` field.

The AIO ring is essentially a circular buffer where the kernel writes the completion reports of the outstanding asynchronous I/O operations. The first bytes of the AIO ring contain an header (a `struct aio_ring` data structure); the remaining bytes store `io_event` data structures, each of which describes a completed asynchronous I/O operation. Because the pages of the AIO ring are mapped in the User Mode address space of the process, the application can check directly the progress of the outstanding asynchronous I/O operations, thus avoiding using a relatively slow system call.

The `io_setup( )` system call creates a new AIO context for the calling process. It expects two parameters: the maximum number of outstanding asynchronous I/O operations, which ultimately determines the size of the AIO ring, and a pointer to a variable that will store a handle to the context; this handle is also the base address of the AIO ring. The `sys_io_setup( )` service routine essentially invokes `do_mmap( )` to allocate a new anonymous memory region for the process that will contain the AIO ring (see the section "[Allocating a Linear Address Interval](#)" in [Chapter 9](#)), and creates and initializes a `kioctx` object describing the AIO context.

Conversely, the `io_destroy( )` system call removes an AIO context; it also destroys the anonymous memory region containing the corresponding AIO ring. The system call blocks the current process until all outstanding asynchronous I/O operations are complete.

16.4.1.2. Submitting the asynchronous I/O operations

To start some asynchronous I/O operations, the application invokes the `io_submit( )` system call. The system call has three parameters:

`ctx_id`

The handle returned by `io_setup( )`, which identifies the AIO context

`iocbpp`

The address of an array of pointers to descriptors of type `iocb`, each of which describes one asynchronous I/O operation

`nr`

The length of the array pointed to by `iocbpp`

The `iocb` data structure includes the same fields as the POSIX `aio` descriptor (`aio_fildes`, `aio_buf`, `aio_nbytes`, `aio_offset`) plus the `aio_lio_opcode` field that stores the type of the requested operation (typically read, write, or sync).

The service routine `sys_io_submit( )` performs essentially the following steps:

1. Verifies that the array of `iocb` descriptors is valid.
2. Searches the `kiocx` object corresponding to the `ctx_id` handle in the list rooted at the `ioctx_list` field of the memory descriptor.
3. For each `iocb` descriptor in the array, it executes the following substeps:
 - a. Gets the address of the file object corresponding to the file descriptor stored in the `aio_fildes` field.
 - b. Allocates and initializes a new `kiocb` descriptor for the I/O operation.
 - c. Checks that there is a free slot in the AIO ring to store the completion result of the operation.
 - d. Sets the `ki_retry` method of the `kiocb` descriptor according to the type of the operation (see below).
 - e. Executes the `aio_run_iocb( )` function, which essentially invokes the `ki_retry` method to start the I/O data transfer for the corresponding asynchronous I/O operation. If the `ki_retry` method returns the value `-EIOCBRETRY`, the asynchronous I/O operation has been submitted but not yet fully satisfied: the `aio_run_iocb( )` function will be invoked again on this `kiocb` at a later time (see below). Otherwise, it invokes `aio_complete( )` to add a completion event for the asynchronous I/O operation in the ring of the AIO context.

If the asynchronous I/O operation is a read request, the `ki_retry` method of the corresponding `kiocb` descriptor is implemented by `aio_pread( )`. This function essentially executes the `aio_read` method of the file object, then updates the `ki_buf` and `ki_left` fields of the `kiocb` descriptor (see [Table 16-1](#) earlier in this chapter) according to the value returned by the `aio_read` method. Finally, `aio_pread( )` returns the number of bytes effectively read from the file, or the value `-EIOCBRETRY` if the function determines that not all requested bytes have been transferred. For most filesystems, the `aio_read` method of the file object ends up invoking the `__generic_file_aio_read( )` function. Assuming that the `O_DIRECT` flag of the file is set, this function ends up invoking the `generic_file_direct_IO( )` function, as described in the previous section. In this case, however, the `__blockdev_direct_IO( )` function does not block the current process waiting for the I/O data transfer to complete; instead, the function returns immediately. Because the asynchronous I/O operation is still outstanding, the `aio_run_iocb( )` will be invoked again, this time by the `aio` kernel thread of the `aio_wq` work queue. The `kiocb` descriptor keeps track of the progress of the I/O data transfer; eventually all requested data will be transferred and the completion result will be added to the AIO ring.

Similarly, if the asynchronous I/O operation is a write request, the `ki_retry` method of the `kiocb` descriptor is implemented by `aio_pwrite( )`. This function essentially executes the `aio_write` method of the file object, then updates the `ki_buf` and `ki_left` fields of the `kiocb` descriptor (see [Table 16-1](#)

earlier in this chapter) according to the value returned by the `aio_write` method. Finally, `aio_pwrite( )` returns the number of bytes effectively written to the file, or the value `-EIOCBRETRY` if the function determines that not all requested bytes have been transferred. For most filesystems, the `aio_write` method of the file object ends up invoking the `generic_file_aio_write_nolock( )` function. Assuming that the `O_DIRECT` flag of the file is set, this function ends up invoking the `generic_file_direct_IO( )` function, as above.